ARE SELF-EVOLVING AGENT SKILLS LEARNING SKILLS?

Anonymous Submission

ABSTRACT

Higher post-evolution scores raise an identification question: what improvement is carried by the persistent skill, and what remains useful after the task changes? We examine this question through a diagnosis of SkillEvolver, SPARK, and CoEvoSkills, connecting their construction procedures, completed artifacts, and downstream behavior. Repeated paired evaluations establish source-task utility, controlled variations probe its scope, and skill contents and execution traces explain contrasting reuse outcomes. The evidence separates recurring benchmark gains from reusable decision guidance: positive source-task benefits coexist with sharply different responses to changed instances and requirements. Successful cases preserve criteria for selecting valid results and maintaining task constraints. This analysis connects observed gains to what the skill helps an executor decide, making behavioral persistence and attribution central to evaluating skill learning.

***Index Terms*—** Agent skills, self-evolution, diagnostic evaluation, skill reuse, generalization

1. INTRODUCTION

Agent skills externalize behavior into persistent instructions, scripts, and references that can be inspected and reused without changing model parameters. SkillEvolver, SPARK, and CoEvoSkills construct these artifacts from execution experience through deployment-grounded revision, verified-trajectory distillation, and co-evolutionary verification [1–3]. A higher score after this process raises the question in our title: what has the system learned as a skill?

The final score is produced through several steps. Construction determines which guidance enters the artifact; the executor decides how to use that guidance; and a verifier assesses the resulting output. A skill can preserve a useful decision rule, an instance-specific choice, or instructions that a later executor applies incompletely. These possibilities can yield similar source-task scores while producing different behavior when the input or requirement changes. Locating the improvement therefore requires examining the persistent artifact together with its deployment.

We pursue two connected research objectives. First, we examine whether completed skills provide recurring utility and how far that utility extends under controlled task variation. Second, we investigate which decisions preserved in the skills help explain successful and failed reuse, and how those decisions relate to the feedback used during construction. This keeps the existence and scope of a benefit separate from the explanation of how it arises.

Our empirical diagnosis begins with two standard-task runs per method, pairing each skill condition with a no-skill execution of the same task. We then apply frozen skills to changes in input representation, task instances, and coupled requirements. Finally, contrasting signal-detection and scheduling cases connect the observed outcomes to the guidance used by the executor. The sequence links the score, the conditions under which it persists, and the decisions carried by the artifact.

The central finding is a separation between recurring source-task benefit and reusable behavior. All three methods have positive paired mean gains in both standard-task runs, but this agreement breaks under task variation. SkillEvolver remains positive across all three change types; SPARK and CoEvoSkills lose utility on replacement instances. The cases reveal a corresponding distinction in skill content: finding a plausible signal differs from identifying the required signal, and constructing a feasible schedule differs from preserving the rules of an acceptable repair. Successful reuse accompanies guidance that retains these distinctions.

We contribute an empirical diagnosis of the persistence of completed-skill utility and an analysis of the decision guidance preserved in successful reuse. The repeated outcomes establish utility at deployment; controlled variations reveal its behavioral scope; and artifact and trace evidence connect that scope to reusable decision criteria. Together, these observations address what benchmark gains say about skill learning.

2. PROBLEM FORMULATION

2.1. Skill Revision and Evaluation

At revision round r, an optimizer updates the current persistent skill artifact:

$$s_{r+1} = O(s_r, \tau_r, f_r), \tag{1}$$

where s_r is the current skill, τ_r is an execution trace, and f_r is feedback from execution and evaluation. The operator O denotes the method's complete update pathway, including its native merge, validation, selection, or adoption mechanism. A generated candidate that is rejected or never adopted is not s_{r+1}.

The agent then executes task x with the frozen skill:

$$o_r = A(x, s_r, z_r), \tag{2}$$

where z_r represents model sampling, tool use, and environment state, and o_r is the resulting output artifact or state change. A verifier maps the output to a score:

$$y_r = V(x, o_r). \tag{3}$$

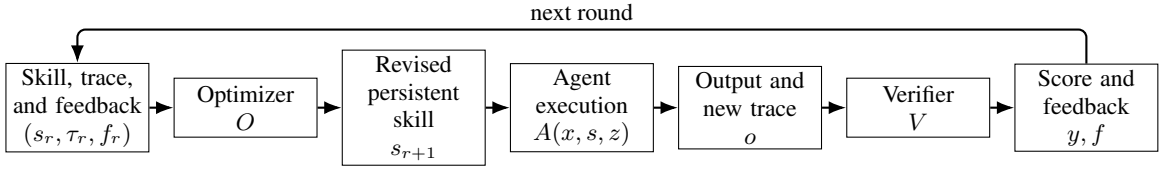


Fig. 1. The skill-evolution lifecycle. Construction produces a persistent artifact; deployment turns its guidance into behavior; verification assigns the observed score. We connect these stages through paired utility, frozen-skill task variation, and inspection of skill use.

2.2. What Does an Observed Gain Identify?

The score $V(x, A(x, s, z))$ depends jointly on the skill, its execution, and the verifier. A skill-versus-no-skill comparison measures the utility of a completed artifact at deployment. A parent-to-final comparison instead measures the change associated with revising an existing artifact. Keeping these contrasts distinct specifies which part of the learning question an evaluation answers.

Our empirical contrast is completed-skill utility. We first examine whether its paired mean is positive in each of two runs on the same task set. We then freeze the source-task skill s_x and apply it to a variant $T_k(x)$, pairing its outcome with a no-skill execution on that variant. The executor can reinterpret inputs and adapt its actions; the skill itself is not revised. Persistence of utility under these changes locates the observed scope of reuse.

Attribution concerns the decisions connecting the artifact to the result. We inspect what the skill tells the executor to retain, recompute, or check, and how that guidance appears in successful and failed executions. Published construction procedures supply an explanation for how such guidance may arise. This separates observed behavior and artifact content from interpretation of the evolution pathway.

3. DOES SKILL UTILITY PERSIST BEYOND THE SOURCE TASK?

We address the existence and scope of completed-skill utility through two contrasts: paired standard-task outcomes across runs, and paired outcomes after changing the task while keeping its source skill frozen.

3.1. Experimental Setup

We evaluate SkillEvolver [1], SPARK [2], and CoEvoSkills [3] on SkillsBench [4], with two runs per method. Each run pairs the method's skill condition with a no-skill execution of the same task. Both runs of each method use the same task set: 81 tasks for SkillEvolver, 84 for SPARK, and 85 for CoEvoSkills.

We report mean verifier reward on a 0–100 scale and the mean within-task difference between the skill and no-skill conditions. For a run with N tasks, the paired gain is

$$\Delta = \frac{100}{N} \sum_{i=1}^{N} \left(r_i^{\text{skill}} - r_i^{\text{no-skill}} \right). \tag{4}$$

We also count tasks with positive, negative, and zero differences.

3.2. Repeated Runs Establish Source-Task Utility

All three methods have positive mean paired gains in both runs (Table 1). SkillEvolver gains 25.61 and 25.02 points, CoEvoSkills gains 10.24 and 9.76, and SPARK gains 1.84 and 6.45. SkillEvolver and CoEvoSkills retain similar gains despite substantial changes in their no-skill scores. The paired comparison reveals recurring utility that absolute skill-condition scores alone would obscure.

The task-level results also differ. SkillEvolver has at least four times as many improvements as regressions in each run; CoEvoSkills improves 17 tasks in each run, with nine and ten regressions. SPARK's first run is nearly balanced, while its second has 13 improvements and five regressions. These results establish recurring source-task utility. We next ask whether the same frozen skills remain useful after the task changes.

Method	Run	N	No skill	Skill	Δ	I/R/T
SkillEvolver	1	81	30.20	55.81	+25.61	27/6/48
	2	81	37.32	62.34	+25.02	26/3/52
SPARK	1	84	17.82	19.66	+1.84	9/8/67
	2	84	14.37	20.81	+6.45	13/5/66
CoEvoSkills	1	85	42.21	52.44	+10.24	17/9/59
	2	85	23.54	33.31	+9.76	17/10/58

Table 1. Two runs per method on SkillsBench. Mean verifier rewards and paired gain Δ use a 0–100 scale. I/R/T counts improved, regressed, and tied tasks.

3.3. Task Variation Tests What Remains Useful

We apply each source-task skill to controlled variants without further evolution. Input-representation changes (P1) alter names, column arrangements, or schemas; task-instance changes (P2) replace the data or instance structure; coupled-requirement changes (P3) alter or add requirements that must be satisfied together. The corresponding decision demands are to remap the input, recompute a solution for a replacement instance, and accommodate changed correctness requirements. A frozen skill must support these adaptations through guidance already present in the artifact.

The complete record contains 222 variants and 886 condition-level results across no skill and the three methods. Table 2 summarizes historical and recovery evaluations for 222 SkillEvolver variants and 221 each for SPARK and CoEvoSkills, pairing each skill condition with the no-skill result on the same variant.

Source-task improvement does not imply useful generalization. All three methods improve mean reward on the source tasks in both runs, but their variant results diverge. SkillEvolver retains a positive mean gain of 11.94 points, SPARK is close to neutral at $+0.65$, and CoEvoSkills falls below the no-skill baseline at -5.01. SkillEvolver also improves more variants than it regresses (47 versus 20), whereas SPARK and CoEvoSkills show the reverse. Thus, agreement on positive source-task utility conceals different responses to changed tasks. Repeated source-task benefit and persistence under variation are distinct observations.

Replacing the instance produces the largest divergence. SkillEvolver remains positive under all three changes, with its largest gain on P2 (+14.56). P2 is also where SPARK and CoEvoSkills decline most (−5.23 and −9.51). SPARK retains positive averages under representation and requirement changes, but loses its advantage when the underlying instance changes. This separates remapping inputs and adjusting requirements from recomputing choices for a new instance. SkillEvolver's positive gains across P1–P3 show the broadest observed persistence of utility, while CoEvoSkills' negative means show that source-task guidance can become counterproductive after the task changes.

A. Overall utility on task variants

Method	N	No skill	Skill	Δ	I/R/T
SkillEvolver	222	41.87	53.81	+11.94	47/20/155
SPARK	221	41.61	42.25	+0.65	32/37/152
CoEvoSkills	221	41.61	36.60	−5.01	26/41/154

B. Utility by task change: $(N)\ \Delta$

Method	P1	P2	P3
SkillEvolver	(77) +11.85	(68) +14.56	(77) +9.72
SPARK	(77) +2.00	(68) −5.23	(76) +4.53
CoEvoSkills	(77) −3.59	(68) −9.51	(76) −2.42

Table 2. Skill utility under task variation: descriptive summary of historical and recovery evaluations. Each method is paired with its same-variant no-skill baseline. P1/P2/P3 denote representation, instance, and requirement changes. Panel B gives variant count N and paired gain Δ; scores and I/R/T follow Table 1.

4. WHAT IN THE SKILL EXPLAINS ITS BEHAVIOR?

The second research objective is to connect the observed outcome to the persistent guidance used during execution. Signal detection and scheduling expose two distinct decisions: selecting the valid candidate and preserving the conditions of a correct repair. We inspect both the rules written into the skills and the behavior observed when they are reused.

4.1. Distinguishing a Candidate from a Valid Answer

In exoplanet detection, the SkillEvolver skill supports successful detection after input columns are rearranged (P1) and after the task requires two signals (P3). The CoEvoSkills skill fails both variants: its use leads to stellar modulation being reported as the target signal, and a signal and its harmonic being treated as two separate signals. Both skills fail the replacement-instance P2 variant.

The difference is visible in what the skills ask the executor to check. SkillEvolver separates candidate discovery, signal diagnosis, and refinement. Its guidance addresses misleading peaks, transit-like durations, and harmonic relationships. CoEvoSkills also includes validation, including phase-fold, odd/even, and residual checks, but its use in these variants does not resolve the misleading candidates. The presence of a checking routine is therefore less informative than whether its criteria distinguish the alternatives that matter on the changed task. SkillEvolver's P1 and P3 successes show this guidance being useful under changed inputs and requirements. Its P2 failure locates a different boundary within the same task family: the retained criteria do not resolve that replacement instance, despite a positive P2 average across the broader evaluation.

4.2. Preserving the Rules That Define a Correct Repair

In scheduling, the SkillEvolver and SPARK skills succeed on all three variants; the CoEvoSkills skill succeeds only on P3. In P1, using CoEvoSkills yields an otherwise feasible schedule that fails a required local-minimality check on the baseline-relative repair. Feasibility checks alone do not capture this local-minimality requirement.

The SkillEvolver skill explicitly retains local minimality and baseline-relative repair rules. These rules specify how an acceptable repair differs from an arbitrary feasible schedule, and remain relevant after the input names or scheduling instance change. The CoEvoSkills skill covers several structural and feasibility checks, yet the failed deployment misses the decisive repair condition. The contrast concerns whether the decisive repair condition is preserved in guidance and enforced during its use. SPARK's success across the same three variants also shows that this form of successful reuse is not exclusive to one method.

4.3. Construction Feedback and Skill Content

The construction procedures help interpret how these decision criteria enter a skill. SkillEvolver compares successful and failed deployments of a candidate skill and patches missing or misleading guidance. Its independent audit also checks reliance on training-instance constants [1]. This exposes failures in using the skill and encourages rules that a later executor can apply to a different input. The scheduling and signal-detection skills reflect that emphasis by retaining criteria for checking a repair or discriminating between candidate signals.

SPARK grounds skill distillation in environment-verified trajectories, preserving execution evidence, checks, and lessons [2]. Its scheduling result shows that such evidence can support successful reuse. Its mixed aggregate profile raises a separate question about abstraction: a verified execution supplies evidence for what worked on the source task, while changed instances require determining which choices should be retained and which should be recomputed. The observed decline on P2 makes that distinction especially relevant to interpreting its results.

CoEvoSkills obtains detailed revision feedback from a co-evolving surrogate verifier, while the ground-truth oracle returns an opaque pass/fail signal [3]. This makes the surrogate's coverage of correctness conditions relevant to the guidance retained in the skill. This provides a plausible account of the scheduling failure: the skill preserves many checks, but successful reuse still depends on a condition those checks did not enforce in that deployment. Across the cases, feedback coverage and abstraction from the source instance are candidate explanations for the different scopes of utility.

5. INTERPRETING GAINS AS EVIDENCE OF SKILL LEARNING

5.1. Utility, Scope, and Attribution Answer Different Questions

Positive paired gains in both runs establish recurring source-task utility of the completed skills. Task variation asks a different question: which parts of that usefulness survive when the original input or solution no longer suffices? The three methods agree on the first observation and diverge on the second. This is the diagnostic result behind the performance differences. A source-task average describes a benefit in one setting; the variation profile reveals what demands the same artifact can accommodate.

Average gain and improvement frequency also reveal different behavior. SPARK's slightly positive variant mean combines more regressions than improvements, so near-zero utility masks opposing changes. SkillEvolver combines a positive mean with more improvements, whereas CoEvoSkills combines a negative mean with more regressions. Pairing these summaries with the cases makes the interpretation concrete: utility depends on whether the guidance helps the executor make the changed task's decisive choices.

5.2. What the Persistent Artifact Must Preserve

The cases suggest that reusable skill learning preserves the relationship between a procedure and the conditions that make its result valid. In signal detection, this relationship distinguishes a target from misleading candidates. In scheduling, it connects repair actions to baseline-relative constraints. A procedure can remain executable while this relationship is lost. Conversely, a skill can guide a new solution by preserving the checks and adaptation rules that still apply.

This connects the diagnosis back to the evolution lifecycle. Feedback supplies evidence about failures; the skill expresses a rule extracted from that evidence; the executor must use it in the new setting; and the verifier records the outcome. The observed cases expose whether the guidance survives this handoff. Construction, adoption, and useful deployment are separate events, even when they contribute to the same reported score.

Evaluation should therefore follow the artifact from its source-task contribution to its changed-task behavior. Repeated paired outcomes show whether utility recurs, frozen variants test which decisions remain applicable, and skill/trace inspection explains where reuse succeeds or breaks. When the claim concerns the improvement produced by revision itself, the corresponding contrast is the actual parent and adopted final artifact. The research target is a persistent change in useful behavior, with the score serving as one observation of that change.

6. RELATED WORK

Learning persistent skills from experience. SkillEvolver, SPARK, and CoEvoSkills make experience persistent through different construction loops: deployment-grounded revision, verified-trajectory distillation, and co-evolutionary verification, respectively [1–3]. These methods motivate our focus on the artifact delivered to a later executor. We study how that artifact's usefulness changes after its source task is varied, and interpret the resulting behavior through its retained guidance.

Evaluating skill utility and use. SkillsBench evaluates skill utility across diverse tasks, while SWE-Skills-Bench studies skill use in software engineering [4, 5]. SkillLearnBench evaluates continual skill generation on real-world tasks [6]. From Raw Experience to Skill Consumption separates experience generation, skill extraction, and downstream consumption [7]. Our study connects these evaluation levels to the identification of skill learning: repeated outcomes characterize completed-skill utility, task changes test its persistence, and skill-level examples explain the decisions behind contrasting outcomes. The unit of interpretation is the persistent artifact in use.

7. CONCLUSION

A higher score becomes informative about skill learning when it is connected to what the persistent artifact helps an executor do. Our repeated evaluations show positive source-task utility for all three methods, while frozen task variations separate the conditions under which that utility persists. SkillEvolver retains positive gains across the three change types; SPARK and CoEvoSkills expose narrower or adverse reuse patterns. The contrasting cases connect successful reuse to criteria for identifying valid answers and preserving repair constraints. These findings distinguish useful completed artifacts, their behavioral scope, and explanations grounded in skill content. They return the learning question to persistent decision guidance: what is retained from experience, when does it remain useful, and how does it shape the next execution?

8. REFERENCES

[1] Genrui Zhang, Erle Zhu, Jinfeng Zhou, Caiyan Jia, and Hongning Wang, "SkillEvolver: Skill Learning as a Meta-Skill," 2026.

[2] Yang Zhou, Zihan Dong, Zhenting Wang, Can Jin, Shiyu Zhao, Bangwei Guo, Difei Gu, Linjun Zhang, Mu Zhou, and Dimitris N. Metaxas, "Evidence Over Plans: Online Trajectory Verification for Skill Distillation," 2026.

[3] Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, Xue Liu, Xiaoxiao Li, and Philip S. Yu, "CoEvoSkills: Self-Evolving Agent Skills via Co-Evolutionary Verification," 2026.

[4] Xiangyi Li, Yimin Liu, Wenbo Chen, Bingran You, Zonglin Di, Yifeng He, Shenghan Zheng, Kyoung Whan Choe, Jiankai Sun, Shuyi Wang, Chujun Tao, Binxu Li, Xuandong Zhao, Hejia Geng, Xiaojun Wu, Junwei Zhou, Xiaokun Chen, Hanwen Xing, Yubo Li, Qunhong Zeng, Di Wang, Yuanli Wang, Roey Ben Chaim, Penghao Jiang, Haotian Shen, Luyang Kong, Xinyi Liu, Runhui Wang, Xuanqing Liu, Jiachen Li, Xin Lan, Yueqian Lin, Wengao Ye, Junwei He, Songlin Li, Yue Zhang, Yipeng Gao, Yijiang Li, Ze Ma, Liqiang Jing, Tianyu Wang, Kaixin Li, Yiqi Xue, Haoran Lyu, Yizhuo He, Yuchen Tian, Shutong Wu, Bowei Wang, Yixuan Gao, Bo Chen, Litong Liu, Sikai Cheng, Jiajun Bao, Shuaicheng Tong, Shuwen Xu, Terry Yue Zhuo, Tinghan Ye, Qi Qi, Miao Li, Longtai Liao, Zelin Tan, Chang Shi, Xilin Tang, Srinath Tankasala, Boqin Yuan, Yaoyao Qian, Jianhong Tu, Chenguang Wang, Yizhou Sun, Wei Wang, Aaron Taylor, Ziyue Yang, Changkun Guan, Zhikang Dong, Xinyu Zhang, Steven Dillmann, Han chung Lee, and Dawn Song, "SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks," 2026.

[5] Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu, "SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering?," 2026.

[6] Shanshan Zhong, Yi Lu, Jingjie Ning, Yibing Wan, Lihan Feng, Yuyi Ao, Leonardo F. R. Ribeiro, Markus Dreyer, Sean Ammirati, and Chenyan Xiong, "SkillLearnBench: Benchmarking Continual Learning Methods for Agent Skill Generation on Real-World Tasks," 2026.

[7] Zisu Huang, Jingwen Xu, Yifan Yang, Ziyang Gong, Qihao Yang, Muzhao Tian, Xiaohua Wang, Changze Lv, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Xue Yang, Dongdong Chen, Xiaoqing Zheng, and Chong Luo, "From Raw Experience to Skill Consumption: A Systematic Study of Model-Generated Agent Skills," 2026.